Experiment No. 6

Inheritance and Types of Inheritance in Java

Aim

- To understand the concept of inheritance in Java.
- To implement different types of inheritance.
- To demonstrate reusability of code using IS–A relationship.
- To understand method overriding in inheritance.

Theory

1. Inheritance in Java

Inheritance is a mechanism by which one class acquires the properties and methods of another class.

Derived Class IS–A Base Class

- Base class (Parent / Super class)
- Derived class (Child / Sub class)

Syntax:

```
class Child extends Parent {
    // code
}
```

2. Advantages of Inheritance

- Code reusability
- Reduced redundancy
- Easy maintenance

3. Types of Inheritance in Java

Type	Description
Single	One child inherits from one parent
Multilevel	Chain of inheritance
Hierarchical	Multiple children inherit from one parent

Note: Multiple inheritance is not supported using classes in Java.

Program 1: Single Inheritance

```
class Number {
    int a, b;

    void accept(int x, int y) {
        a = x;
        b = y;
    }
}

class Sum extends Number {
    void calculate() {
        System.out.println("Sum = " + (a + b));
    }
}

public class SingleInheritanceDemo {
    public static void main(String[] args) {
        Sum s = new Sum();
        s.accept(10, 20);
        s.calculate();
    }
}
```

Program 2: Multilevel Inheritance

```
class Number {
    int n;

    void set(int n) {
        this.n = n;
    }
```

```
}

class Square extends Number {
    int square() {
        return n * n;
    }
}

class Cube extends Square {
    int cube() {
        return n * n * n;
    }
}

public class MultilevelInheritanceDemo {
    public static void main(String[] args) {
        Cube obj = new Cube();
        obj.set(5);
        System.out.println("Square = " + obj.square());
        System.out.println("Cube = " + obj.cube());
    }
}
```

Program 3: Hierarchical Inheritance

```
class Shape {
    int a, b;
}

class Rectangle extends Shape {
    void area() {
        System.out.println("Area of Rectangle = " + (a * b));
    }
}

class Triangle extends Shape {
    void area() {
        System.out.println("Area of Triangle = " + (0.5 * a * b))
            ;
    }
}
```

```
public class HierarchicalInheritanceDemo {
    public static void main(String[] args) {
        Rectangle r = new Rectangle();
        r.a = 10;
        r.b = 5;
        r.area();

        Triangle t = new Triangle();
        t.a = 10;
        t.b = 5;
        t.area();
    }
}
```

Program 4: Method Overriding

```
class Bank {
    double getInterestRate() {
        return 5.0;
    }
}

class SBI extends Bank {
    double getInterestRate() {
        return 6.5;
    }
}

public class MethodOverridingDemo {
    public static void main(String[] args) {
        Bank b = new SBI();
        System.out.println("Interest Rate = " + b.getInterestRate
            ());
    }
}
```

Laboratory Tasks

Task 1: Inheritance with Array of Objects

Design a base class `Person` with data members:

- person ID
- person name

Derive a class `Student` from `Person` by adding:

- marks in three subjects

Create an array of `Student` objects to store details of `N` students ($N \geq 3$).
For all students:

- compute total and average marks
- identify and display the student with the highest average

Note: Student details must be accessed through inherited members of the base class.

Task 2: Multilevel Inheritance

Design a multilevel inheritance structure as follows:

- Base class `Number` stores an integer value.
- Derived class `FactorAnalysis` computes and displays:
  - count of factors of the number
  - sum of factors
- Further derived class `NumberClassification` determines whether the number is:
  - Perfect
  - Deficient
  - Abundant

The number must be initialized in the base class and passed through the inheritance chain using methods.

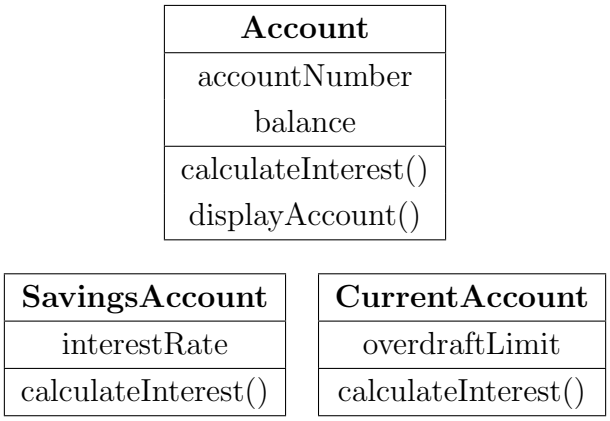
Expected Output: Display the number, its factors, sum of factors, and final classification.

Number Classification:

Number Type	Condition
Perfect	Sum of proper divisors = Number
Deficient	Sum of proper divisors < Number
Abundant	Sum of proper divisors > Number

Task 3: Application-Oriented Design using Inheritance

Study the following class diagram carefully and implement the system using inheritance.



Problem Statement:

- `SavingsAccount` and `CurrentAccount` must inherit from `Account`.
- Interest calculation logic must differ in both derived classes.
- Account details must be displayed using overridden methods.

Deliverables:

- Class definitions as per diagram
- Implementation of overridden methods

Post-Lab Questions

1. What is inheritance?
2. Difference between single and multilevel inheritance.
3. Why multiple inheritance is not supported in Java using classes?
4. What is method overriding?
5. Difference between method overloading and overriding.

Learning Outcomes

After completing this experiment, students will be able to:

- Apply inheritance concepts in Java.
- Identify different types of inheritance.

- Implement method overriding.
- Design programs using IS–A relationship.